IMPLEMENTATION

1. Source File Overview

All pass logic lives in a single file: lib/EnergyPass.cpp (~400 lines, C++17). The file is structured in eight self-contained sections separated by LLVM-style banner comments. There are no header files: the pass is entirely defined and registered in this one translation unit. The lib/CMakeLists.txt builds it as a shared library (MODULE target) using LLVM's add_llvm_pass_plugin() macro, which sets the correct -fno-rtti / -fvisibility=hidden flags automatically on each platform.

2. LLVM Headers Used

Header	Why it is needed
llvm/IR/Instructions.h	isa<LoadInst>, isa<StoreInst>, BranchInst, CallInst — used by classify()
llvm/IR/CFG.h	predecessors() and successors() iterators — used by computeDominators()
llvm/IR/DebugInfoMetadata.h	DIScope::getFilename() — extracts source filename from DebugLoc scope
llvm/IR/DebugLoc.h	DebugLoc — maps instructions back to source file + line + column
llvm/IR/DiagnosticInfo.h	OptimizationRemarkAnalysis — the DiagnosticInfo subclass emitted via LLVMContext
llvm/IR/PassManager.h	PassInfoMixin<T>, FunctionAnalysisManager, PreservedAnalyses
llvm/Passes/PassBuilder.h	PassBuilder — needed for registerPipelineParsingCallback
llvm/Plugins/PassPlugin.h	LLVM_PLUGIN_API_VERSION, llvmGetPassPluginInfo() entry point
llvm/Support/JSON.h	json::parse(), getAsObject(), getAsNumber() — energy model loading
llvm/TargetParser/Triple.h	Triple::getArch() — detects aarch64 vs x86_64 from module target triple

3. Pass Registration

The plugin exposes one C-linkage symbol: llvmGetPassPluginInfo(). This returns an llvm::PassPluginLibraryInfo struct containing the LLVM plugin API version, the plugin name string "EnergyPass", the LLVM version string, and a registration callback lambda. The callback

is invoked by opt during plugin load and calls PB.registerPipelineParsingCallback to bind the name string "energy-pass" to a factory that constructs EnergyPass and adds it to the FunctionPassManager. No analysis passes are registered — this is the key design decision that avoids the macOS duplicate-symbol crash.

The factory reads ENERGY_MODEL_PATH from the environment (or defaults to "data/energy_model.json") and creates a shared_ptr<ModuleAccumulator> that is shared across all EnergyPass instances in the same pipeline run. The accumulator aggregates function-level totals so a module-level summary could be printed after all functions are processed (currently emitted implicitly via the per-function reports).

4. EnergyPass::run — Step by Step

Step 1: *Guard:* F.isDeclaration() returns immediately with PreservedAnalyses::all(). External declarations have no body to analyse and no instructions to classify.

Step 2: *Loop depth computation:* computeLoopDepths(F) is called first, before any model loading, because it only needs the CFG (successors/predecessors) and the dominator map it computes internally. Returns std::map<const BasicBlock*, unsigned> mapping each block to its nesting depth.

Step 3: *Architecture detection:* detectArch(*M) calls Triple(M.getTargetTriple()) and dispatches on Triple::getArch(). Returns "aarch64" for Triple::aarch64 and Triple::aarch64_be, "x86_64" for Triple::x86_64 and Triple::x86, and defaults to "aarch64" for anything else (embedded targets are closer to ARM profiles).

Step 4: *Model loading:* loadModel(modelPath, arch) opens the JSON file via MemoryBuffer::getFile, parses it with json::parse, navigates root→architectures→arch, and copies key/value pairs into EnergyModel::costs. Three levels of fallback: file not found, JSON parse error, arch key missing , each prints a warning and calls loadDefaults().

Step 5: *Block loop:* Iterates every BasicBlock in the function in IR order. analyseBlock() classifies each instruction, accumulates category counts and static energy into a BlockResult, captures the first DebugLoc encountered (used later for block-level remarks), and multiplies static energy by the loop frequency weight to get weighted energy. BlockResult is pushed into FunctionResult::blocks.

Step 6: *Function accumulation:* After the block loop, FunctionResult holds total instrCount, staticEnergy, weightedEnergy, per-category counts, and per-category weighted costs. The last field (weightedCosts[cat]) accumulates cnt * model.cost(cat) * block.frequencyWeight for each block. This is what drives the percentage breakdown in the function summary table.

Step 7: *Output:* printFunctionResult() always runs (stderr). emitFunctionRemark() fires only when a DebugLoc exists on the entry block's first instruction. emitBlockRemark() fires only for blocks with loopDepth > 0 and a valid firstDebugInstr, the hot blocks most relevant to the repair loop.

5. Remarks Implementation Detail

OptimizationRemarkAnalysis is constructed with four arguments: pass name string ("energy"), remark name string ("FunctionEnergy" or "HotBlock"), a DebugLoc, and a pointer to the BasicBlock the remark is attributed to. The remark message is assembled into a std::string via raw_string_ostream and appended with the stream-insertion operator (R << msg). Calling Fn.getContext().diagnose(R) dispatches the remark through LLVMContext's diagnostic handler, which checks whether the remark's pass name matches the -pass-remarks-analysis = filter and, if so, writes it to the YAML file specified by -pass-remarks-output=. This path entirely bypasses FAM, LLVMContext is always available from any instruction or function without needing to query the analysis manager.

6. Build System Details

- **CMakeLists.txt (root):** find_package(LLVM REQUIRED CONFIG) locates the LLVM CMake package at the path set in CMakeUserPresets.json (default: build/lib/cmake/llvm). Two include_directories() calls are required: one for llvm-project/llvm/include (source headers — IR, Support, TargetParser) and one for build/include (generated headers — PassPlugin.h, intrinsic tables, config.h). Missing either directory causes compilation failures on specific headers (PassPlugin.h lives only in the build tree). CMAKE_CXX_STANDARD is set to 17 to match the LLVM build.
- **CMakePresets.json:** Two named configure presets: mac-arm64 (Ninja generator, RelWithDebInfo, CMAKE_OSX_ARCHITECTURES=arm64, LLVM_DIR from sourceDir/build/lib/cmake/llvm) and windows-msvc (Ninja generator, RelWithDebInfo,

CMAKE_CXX_COMPILER=cl). RelWithDebInfo is used instead of Release so that the plugin itself retains debug symbols — useful when debugging the pass, and required for the pass to emit correct DebugLoc metadata in its own diagnostic messages.

- **lib/CMakeLists.txt:** add_llvm_pass_plugin(EnergyPass EnergyPass.cpp) is the single build rule. The LLVM CMake macro sets -fno-rtti (matching LLVM's own build), -fvisibility=hidden (prevents accidental symbol export on Linux/macOS), and links against the LLVM component libraries detected by find_package. No explicit target_link_libraries call is needed — the macro handles LLVM component linking automatically.

7. Key Implementation Constraints

- **No FAM queries —** EnergyPass::run() takes a FunctionAnalysisManager& parameter but never calls FAM.getResult<>() on any analysis. Queries through FAM in an out-of-tree plugin cause duplicate-symbol linker errors on macOS because the plugin re-registers analyses that are already registered in the opt binary.
- **No global mutable state —** All mutable state lives inside EnergyPass instances (modelPath, shared ModuleAccumulator) or local variables in run(). The dominator map and loop depth map are computed fresh per-function and discarded after use. This is safe for multi-threaded opt -j N pipelines.
- **PreservedAnalyses::all() —** The pass is an analysis-only pass: it reads IR but modifies nothing. Returning PreservedAnalyses::all() tells the pass manager that no subsequent pass needs to re-run its analyses because this pass left the IR unchanged.
- **isRequired() = true —** Returning true from the static isRequired() method prevents the pass manager from skipping EnergyPass even when it believes the pass is redundant. Without this, opt may silently skip the pass on functions it judges to be trivial.